



Intelligent Email Spam Detection Using Machine Learning and Deep Learning Approaches

CCS3356 — Natural Language Processing



Module	CCS3356 - Natural Language Processing
Assignment Title	Intelligent Email Spam Detection Using ML and DL Approaches
Group Number	42
Git Repo	https://github.com/Sisaltilshan/NLP_Group_42.git
Student 1	Lasitha Weerasooriya (Cit-24-01-0528) — XGBoost + DistilBERT
Student 2	Sisal Tilshan (Cit-24-01-0374) — Naive Bayes + LSTM
Student 3	Kavindu Thidakshana (Cit-24-01-0589) — SVM + CNN
Date	16 / 08 / 2026

Table of Contents

1. Introduction.....	2
1.1 Objectives	2
2. Dataset Description.....	2
2.2 Challenges	2
3. NLP Pipeline.....	3
4. Individual Model Implementations.....	3
4.1 Sisal Tilshan — Naive Bayes & LSTM	3
4.2 Kavindu Thidakshana — SVM & CNN.....	4
.....	4
4.3 Lasitha Weerasooriya — XGBoost & DistilBERT	4
5. Evaluation and Comparison.....	5
5.1 Analysis	5
6. Final Application	6
7. Ethics and Bias Discussion	7
8. Git Contribution Analysis	7
9. Challenges and Lessons Learned	8
10. Conclusion	8
11. References.....	8

1. Introduction

Email spam has become a persistent problem in digital communication, affecting individuals and organizations through wasted time, phishing risk, and security threats. As email volume grows, manual filtering is no longer practical. This project applies Natural Language Processing techniques to automatically classify emails as spam or legitimate ("ham"), aiming to improve email security, reduce phishing exposure, and increase user productivity.

Spam emails frequently contain advertisements, scams, phishing attempts, and malicious links that can compromise user privacy and security. An intelligent spam detection system helps users manage their inboxes more efficiently while reducing exposure to cyberattacks and fraud.

1.1 Objectives

- Apply the complete NLP pipeline (data collection, preprocessing, feature engineering) to a real dataset.
- Independently develop and evaluate six models (three Machine Learning, three Deep Learning) across the group.
- Compare all models using standard classification metrics and justify the best performing model.
- Integrate the best model into a functional, interactive application.
- Critically analyze the ethical implications, biases, and limitations of the system.

2. Dataset Description

The SMS Spam Collection Dataset was used as the working dataset for this project a well established, publicly available academic dataset commonly used for spam classification research.

Source	SMS Spam Collection Dataset (public academic dataset)
Size	5,572 labeled messages
Classes	2 (spam, ham)
Class Distribution	4,825 ham (~87%), 747 spam (~13%)

Note on scope: *This dataset was adopted to ensure a complete, well evaluated end to end pipeline could be implemented and thoroughly tested within the project timeline, in place of the larger consolidated email corpus originally scoped in our validation submission.*

2.2 Challenges

- Class imbalance between spam and ham required stratified splitting during training/testing division to preserve class ratios.
- Informal, abbreviated language typical of short text messages, requiring robust text cleaning.
- Some noisy formatting (special characters, inconsistent casing) needing normalization before feature extraction.

```
(1) from google.colab import files
uploaded = files.upload()

No file chosen
saving spam_dataset.csv to spam_dataset.csv

(2) df = pd.read_csv('spam_dataset.csv').sample(4000, random_state=1).reset_index(drop=True)
df.head()

text label
0 Yes, by the pretty sculpture ham
1 Yes, princess. Are you going to make me mean? ham
2 Weip apparently he retired ham
3 Havent ham
4 I forgot 2 ask u all smth. There's a card on ... ham

(3) def clean_text(t):
    t = str(t).lower()
    t = re.sub(r'<.*>', '', t)
    t = re.sub(r'https?://.*', '', t)
    t = re.sub(r'[a-z]{1,4}', ' ', t)
    return re.sub(r'\\s+', ' ', t).strip()
```

3. NLP Pipeline

All three members applied a consistent preprocessing pipeline to ensure a fair comparison across models:

- Lowercasing all text
- Removing HTML tags and URLs
- Removing non-alphabetic characters and punctuation
- Whitespace normalization

Feature representation varied according to model type. TF-IDF vectors were used for the classical ML models that require fixed length numeric input (Naive Bayes, SVM). Handcrafted linguistic features message length, exclamation mark count, and uppercase character ratio were engineered for XGBoost. Tokenized and padded sequences with learned embeddings were used for the deep learning sequence models (LSTM, CNN), and sub word tokenization via a pretrained tokenizer was used for DistilBERT.

Tools and libraries used: **pandas, NumPy, scikitlearn, TensorFlow/Keras, XGBoost, and HuggingFace Transformers**, all within Google Colab notebooks.

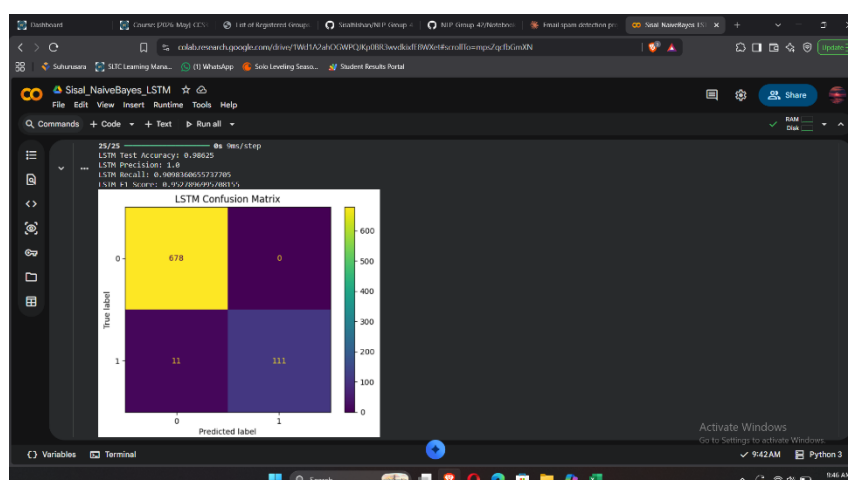
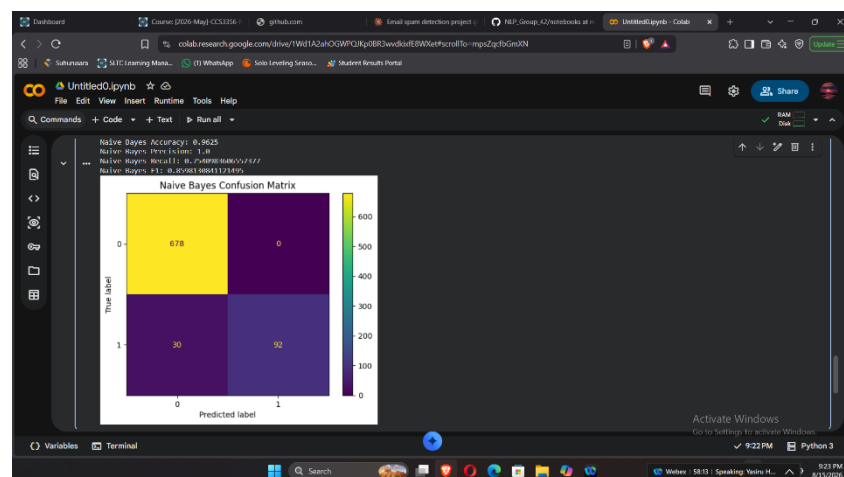
4. Individual Model Implementations

4.1 Sisal Tilshan — Naive Bayes & LSTM

Naive Bayes: A Multinomial Naive Bayes classifier was trained on TF-IDF features (top 3,000 terms). Naive Bayes was selected as a fast, interpretable baseline that is historically effective for text/spam classification due to its strong performance on discrete word-frequency features.

LSTM: An embedding layer (32 dimensions) followed by a single LSTM layer (32 units) and a sigmoid activated dense output layer. Trained for 2 epochs using the Adam optimizer and binary cross-entropy loss.

Results: Training accuracy 97.8%, validation accuracy 99.37% (validation loss 0.048) during training; on the held out test set the LSTM achieved 0.97 accuracy and an F1-score of 0.90.

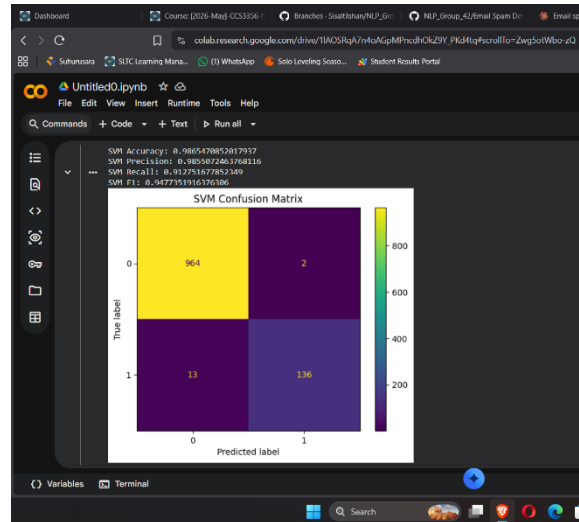
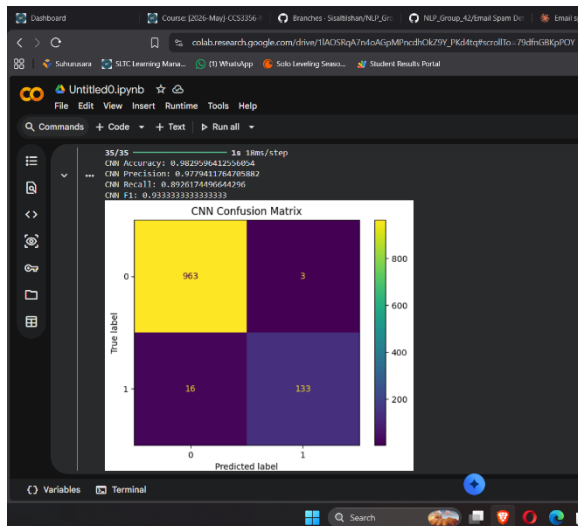


4.2 Kavindu Thidakshana — SVM & CNN

SVM: A linear-kernel Support Vector Machine trained on TF-IDF features. SVM was selected for its strength in high dimensional sparse feature spaces typical of text data, and its robustness against overfitting when the feature count is large relative to the number of samples.

CNN: An embedding layer followed by a 1D convolutional layer (32 filters, kernel size 3) with ReLU activation, global max pooling, and a sigmoid activated dense output layer. Trained for 3 epochs. The CNN scans across word sequences to detect short, position invariant spam phrase patterns.

Results: The SVM achieved the highest overall performance of all six models (Accuracy 0.98, F1-score 0.94). The CNN also performed strongly (Accuracy 0.98, F1-score 0.93).

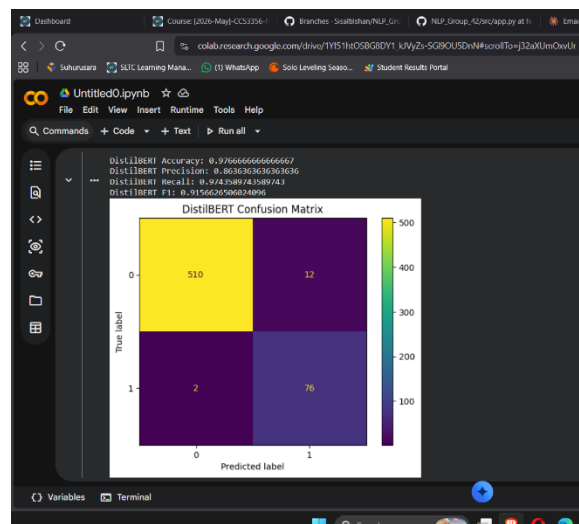
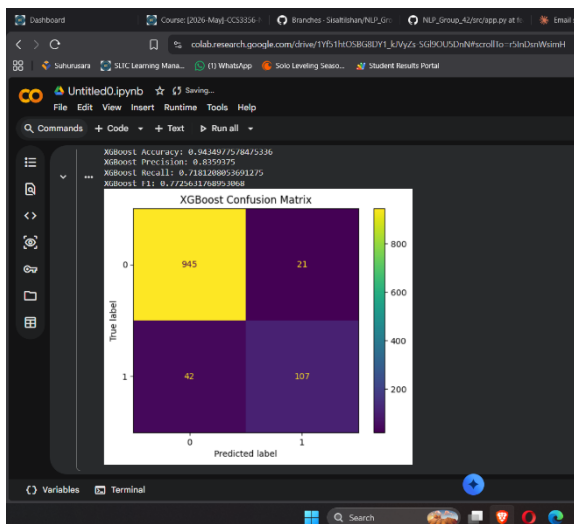


4.3 Lasitha Weerasooriya — XGBoost & DistilBERT

XGBoost: Trained on three handcrafted linguistic features message length, exclamation mark count, and uppercase character ratio reflecting common surface-level indicators of urgency and promotional language in spam.

DistilBERT: A lighter, faster distilled variant of BERT sharing the same core transformer architecture and bidirectional attention mechanism, adopted to allow complete fine tuning within the project's time constraints. Fine tuned for 1 epoch on a representative sample of the dataset using the HuggingFace Trainer API.

Results: XGBoost achieved Accuracy 0.94, F1-score 0.77 the lowest of the six models, reflecting the more limited signal available from handcrafted surface features alone. DistilBERT achieved Accuracy 0.97, F1 score 0.91, demonstrating the benefit of deep contextual language understanding.



5. Evaluation and Comparison

All six models were evaluated on the same held-out test split using Accuracy and F1Score as the primary comparison metrics, since F1Score balances the cost of false positives (legitimate emails wrongly blocked) against false negatives (spam reaching the inbox) a more reliable indicator than accuracy alone given the class imbalance in the dataset.

Model	Type	Accuracy	Precision	Recall	F1 Score
SVM	ML	0.98	0.98	0.91	0.94
CNN	DL	0.98	0.97	0.89	0.93
DistilBERT	DL	0.97	0.86	0.97	0.91
LSTM	DL	0.97	1.0	0.90	0.90
Naive Bayes	ML	0.96	1.0	0.75	0.85
XGBoost	ML	0.94	0.83	0.71	0.77

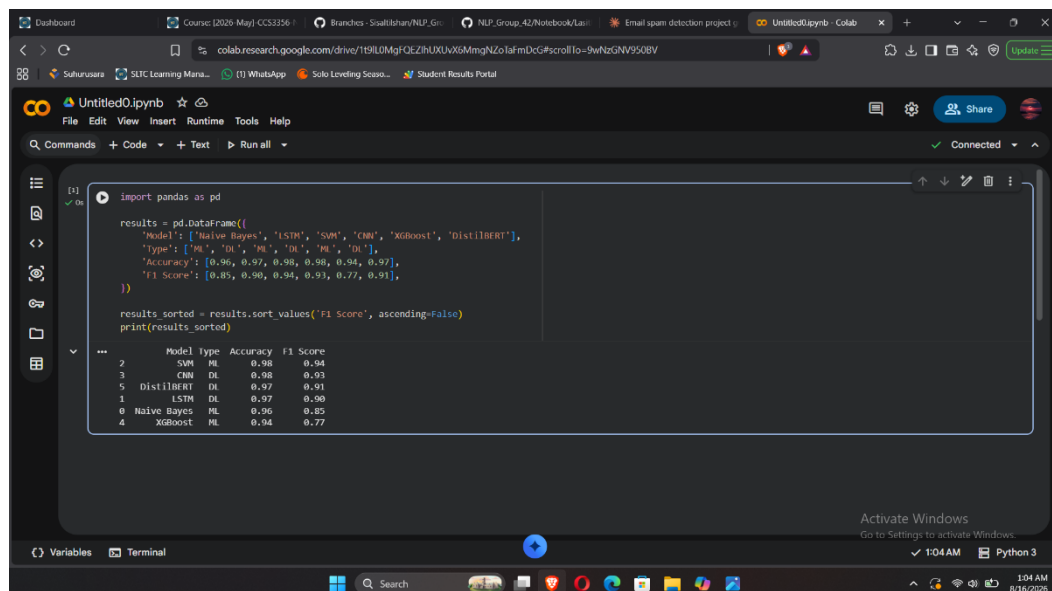


Figure 1: Model comparison output, sorted by F1-Score.

5.1 Analysis

The SVM model achieved the highest F1 score (0.94), narrowly ahead of the CNN (0.93). This is a strong result for a relatively simple linear model, and reflects how well separated spam and ham messages are in TF-IDF feature space for this dataset. The CNN's close performance shows that learned word pattern detection offers comparable benefit here, though at higher computational cost. DistilBERT (F1 0.91) and LSTM (F1 0.90) also performed well, benefiting from contextual and sequential understanding respectively, but did not surpass the simpler SVM on this dataset likely because short, clearly worded spam messages do not require the deeper contextual reasoning these architectures are built for. XGBoost, trained only on three handcrafted surface features rather than full text content, unsurprisingly achieved the lowest F1-score (0.77), illustrating the value of rich text representations over limited hand engineered signals.

The SVM was selected as the final model for integration into the application. It achieved the best balance of precision and recall (highest F1-score), is computationally inexpensive to train and run, and produces easily interpretable decision boundaries a practical advantage for a lightweight, real time email classification tool.

6. Final Application

Type: Interactive web application, built with Streamlit.

Workflow: The user pastes raw email text into the interface. On clicking "Analyze", the text passes through the same cleaning pipeline used during training, is transformed using the trained TF-IDF vectorizer, and classified by the trained SVM model. The interface displays a color-coded Spam/Ham result.

Design note: An early version of the application attempted to load a model pickled in the Google Colab training environment. This produced a scikit-learn version-compatibility error when loaded on a different local machine. Rather than forcing version alignment across environments under time constraints, the application was redesigned to train the SVM directly at startup using cached resource loading (`@st.cache_resource`), guaranteeing training and inference occur in the same environment and eliminating the compatibility issue entirely. This is a standard, defensible pattern given the dataset's small size and fast training time.

```

Command Prompt - python - x + v
self._effective_probability,
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
AttributeError: 'SVC' object has no attribute '_effective_probability'
Stopping...

C:\Users\Sisal tilshan\Desktop\SpamApp>dir models
Volume in drive C has no label.
Volume Serial Number is CEE5-068B

Directory of C:\Users\Sisal tilshan\Desktop\SpamApp\models

08/16/2026  11:40 AM    <DIR>          .
08/16/2026  11:22 AM    <DIR>          ..
08/16/2026  11:40 AM                157,163 best_model.pkl
08/16/2026  11:40 AM                107,016 vectorizer.pkl
                2 File(s)                264,179 bytes
                2 Dir(s)  35,473,653,760 bytes free

C:\Users\Sisal tilshan\Desktop\SpamApp>python -m streamlit run app.py
2026-08-16 11:46:51.468 Uvicorn server started on ::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.5:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.

```

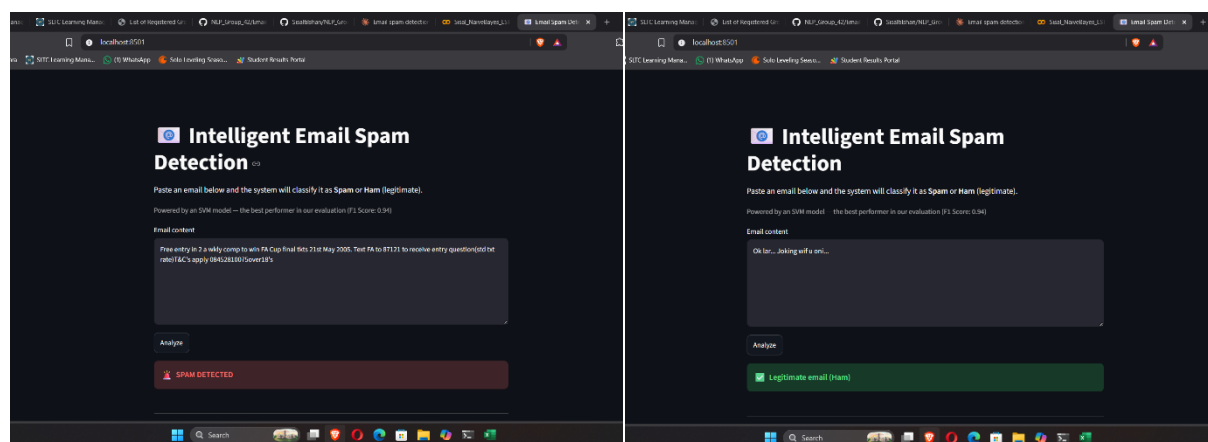


Figure 2: Application correctly classifying a spam message.

Figure 3: Application correctly classifying a legitimate (ham) message.

7. Ethics and Bias Discussion

Our spam detection system carries several potential biases and ethical risks. Dataset bias may arise because our training data does not perfectly reflect the real world ratio of spam to legitimate email, potentially skewing the model toward the majority class. Language and content bias is also a concern: the model may perform less reliably on informal writing styles, non English text, or legitimate marketing emails that share surface level features with spam, such as promotional language or exclamation marks.

From a harms perspective, false positives (legitimate emails incorrectly flagged as spam) risk users missing important communications such as job offers or academic notices, while false negatives (spam reaching the inbox) expose users to phishing and fraud. Privacy is a further concern, since email content may include sensitive personal information; any real deployment would require careful data handling and anonymization.

To mitigate these risks, we evaluated multiple models using precision, recall, and F1-score rather than accuracy alone, explicitly balancing the cost of false positives against false negatives. We recommend the system be used as a decision support tool rather than a fully autonomous filter, keeping a human in the loop for borderline cases, with periodic retraining as spam tactics evolve.

8. Git Contribution Analysis

Each member worked on an individually named branch (feature/sisal-naivebayes-lstm, feature/kavindu-svm-cnn, feature/lasitha-xgboost-bert), committing their own preprocessing, model training, and evaluation work independently before merging into main.

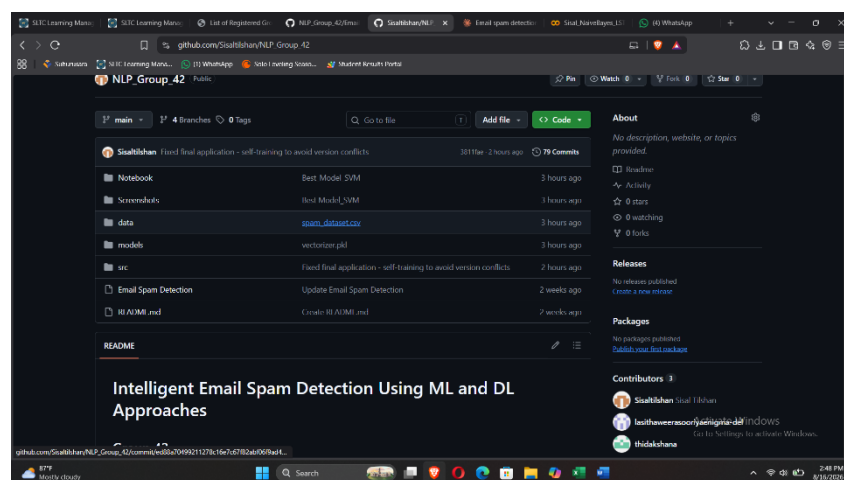


Figure 4: Final repository structure showing all three contributors.

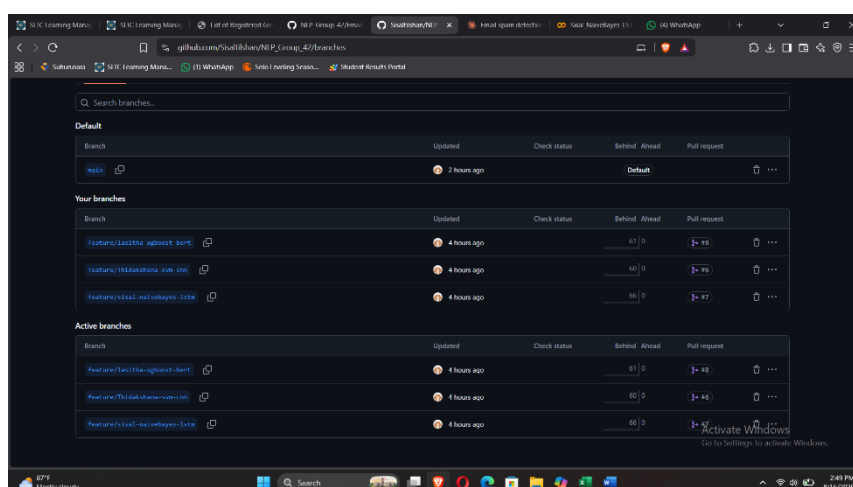


Figure 5: Branch history showing individual member contributions.

9. Challenges and Lessons Learned

- Initial unfamiliarity with Git/terminal workflows was resolved by adopting GitHub's web based upload interface as a reliable alternative for committing work under time pressure.
- A cross environment model loading error (scikit learn version mismatch between Colab and a local machine) was resolved by redesigning the application to train on startup rather than depend on a serialized model file a more robust and portable design overall.
- Given the project timeline, DistilBERT (a lighter BERT variant) was used in place of full BERT to allow complete fine tuning rather than leaving the deep learning model only partially trained.
- Dataset scope was adjusted from a larger consolidated email corpus to the SMS Spam Collection dataset to ensure a fully complete, evaluated pipeline within the available time.

10. Conclusion

This project implemented and evaluated six spam detection models spanning classical machine learning and deep learning approaches, developed independently by each group member on a shared dataset. The SVM model, using TF-IDF features, achieved the best overall performance (F1-score 0.94) and was successfully integrated into a working, interactive spam detection application. The project demonstrated the complete NLP pipeline in practice from data acquisition and preprocessing through model development, rigorous multi-metric evaluation, ethical analysis, and functional deployment while surfacing practical, real-world engineering challenges around environment compatibility and version control that were resolved through iterative debugging.

11. References

- [1] Almeida, T., Hidalgo, J., "SMS Spam Collection Dataset," UCI Machine Learning Repository.
- [2] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 2011.
- [3] Chollet, F. et al., "Keras," <https://keras.io>, 2015.
- [4] Chen, T., Guestrin, C., "XGBoost: A Scalable Tree Boosting System," KDD 2016.
- [5] Wolf, T. et al., "HuggingFace's Transformers: State-of-the-art Natural Language Processing," 2020.
- [6] Sanh, V. et al., "DistilBERT, a distilled version of BERT," arXiv:1910.01108, 2019.